In [46]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, class
```

In [48]:
```
df = pd.read_csv("insurance.csv")
df.head(len(df))
```

Out[48]:

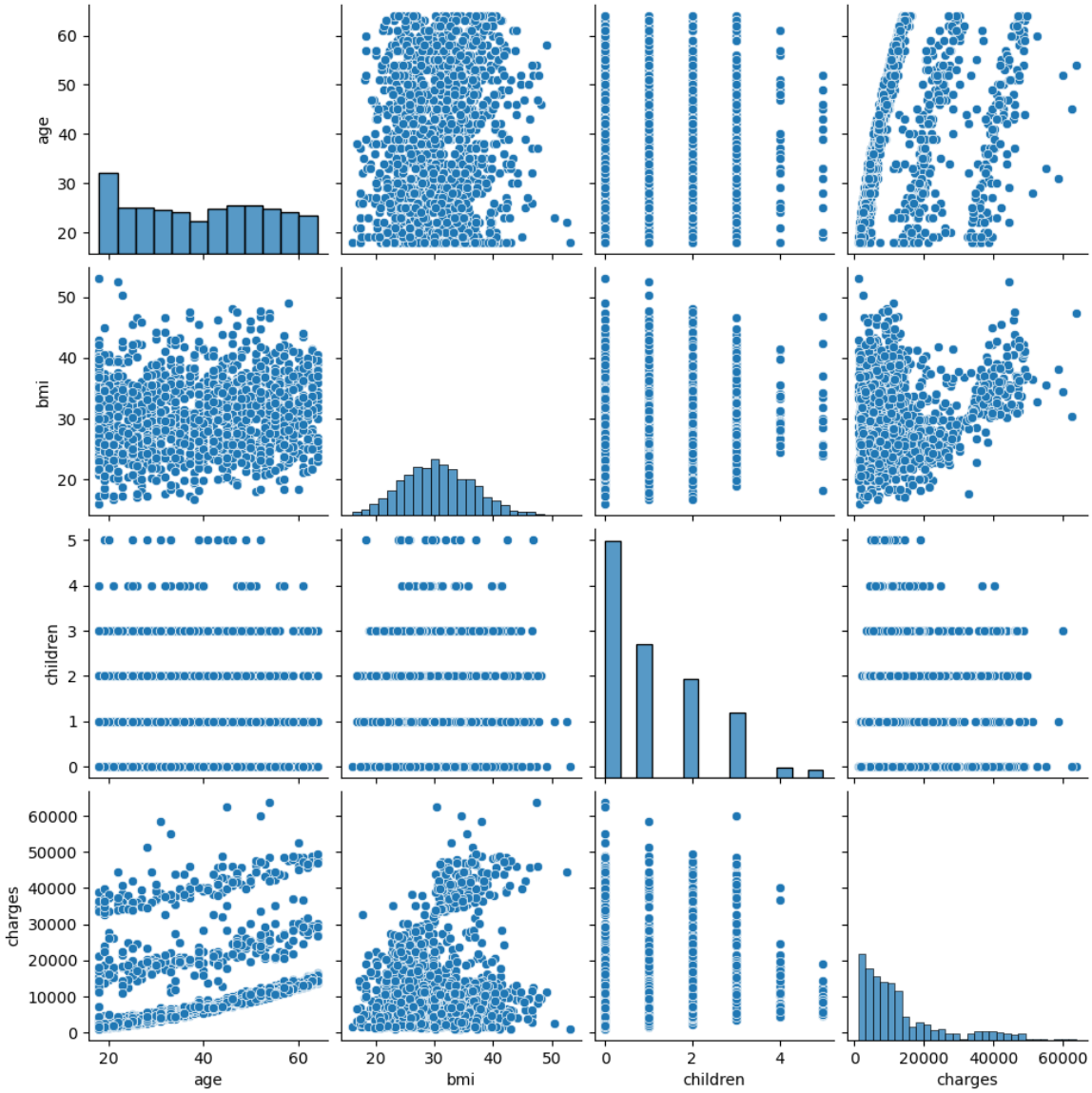
	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

1338 rows × 7 columns

In [50]:
```
print(df.info())
print(df.describe())

sns.pairplot(df)
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
                age          bmi     children       charges
count  1338.000000  1338.000000  1338.000000   1338.000000
mean     39.207025    30.663397     1.094918  13270.422265
std      14.049960     6.098187     1.205493  12110.011237
min      18.000000    15.960000     0.000000   1121.873900
25%      27.000000    26.296250     0.000000   4740.287150
50%      39.000000    30.400000     1.000000   9382.033000
75%      51.000000    34.693750     2.000000  16639.912515
max      64.000000    53.130000     5.000000  63770.428010
```



In [52]:
```
# Encode categorical variables
le = LabelEncoder()

df['sex'] = le.fit_transform(df['sex'])
df['smoker'] = le.fit_transform(df['smoker'])
df['region'] = le.fit_transform(df['region'])

# Convert regression target → classification
df['charges_category'] = pd.qcut(df['charges'], q=3, labels=[0,1,2]

df.head()
```

Out[52]:

	age	sex	bmi	children	smoker	region	charges	charges_catego
0	19	0	27.900	0	1	3	16884.92400	
1	18	1	33.770	1	0	2	1725.55230	
2	28	1	33.000	3	0	2	4449.46200	
3	33	1	22.705	0	0	1	21984.47061	
4	32	1	28.880	0	0	1	3866.85520	

In [54]:
```
# Target variable
y = df['charges_category']

# Features
X = df[['age', 'sex', 'bmi', 'children', 'smoker', 'region']]

print(X.columns)
```

```
Index(['age', 'sex', 'bmi', 'children', 'smoker', 'region'], dtype='
object')
```

In [56]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [58]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [60]:
```
models = {}

# MLP Classifier (Neural Network)
mlp = MLPClassifier(hidden_layer_sizes=(32,16),
                    activation='relu',
                    solver='adam',
                    max_iter=500,
                    random_state=42)

mlp.fit(X_train_scaled, y_train)
models['MLP Classifier'] = mlp
```

```
# Logistic Regression
lr = LogisticRegression(max_iter=500)
lr.fit(X_train_scaled, y_train)
models['Logistic Regression'] = lr


# Decision Tree
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)
models['Decision Tree'] = dt


# Random Forest (Best for accuracy)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
models['Random Forest'] = rf
```

```
/opt/anaconda3/lib/python3.12/site-packages/sklearn/neural_network/_
multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimiz
er: Maximum iterations (500) reached and the optimization hasn't con
verged yet.
  warnings.warn(
```

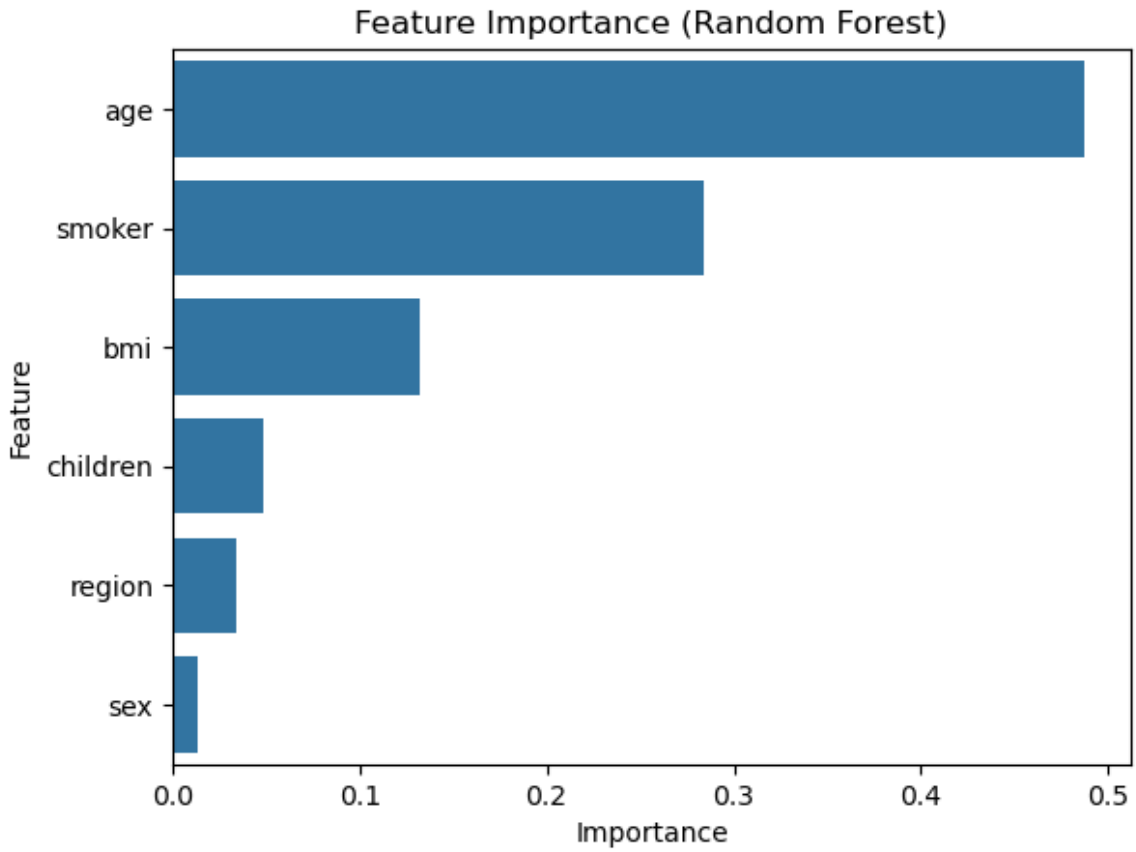
In [62]:

```
rf_model = models['Random Forest']

importances = rf_model.feature_importances_
features = X.columns

importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title("Feature Importance (Random Forest)")
plt.show()
```



In [64]:
```
results = []

for name, model in models.items():

    if name in ['Decision Tree', 'Random Forest']:
        y_pred = model.predict(X_test)
    else:
        y_pred = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, y_pred)

    results.append([name, acc])

# Convert to DataFrame
results_df = pd.DataFrame(results, columns=['Model', 'Accuracy'])
results_df.sort_values(by='Accuracy', ascending=False)
```

Out[64]:

	Model	Accuracy
0	MLP Classifier	0.895522
2	Decision Tree	0.888060
3	Random Forest	0.888060
1	Logistic Regression	0.843284

In [66]:
```
for name, model in models.items():

    if name in ['Decision Tree', 'Random Forest']:
        y_pred = model.predict(X_test)
```

```
    else:
        y_pred = model.predict(X_test_scaled)

    print(f"\n🔷 {name} Performance")
    print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test,
```

```
🔷 MLP Classifier Performance
Accuracy : 0.8955
Confusion Matrix:
 [[87  0  2]
 [ 2 79  8]
 [ 8  8 74]]
Classification Report:
               precision    recall  f1-score   support

           0       0.90      0.98      0.94        89
           1       0.91      0.89      0.90        89
           2       0.88      0.82      0.85        90

    accuracy                           0.90       268
   macro avg       0.90      0.90      0.89       268
weighted avg       0.90      0.90      0.89       268


🔷 Logistic Regression Performance
Accuracy : 0.8433
Confusion Matrix:
 [[83  6  0]
 [ 2 87  0]
 [ 9 25 56]]
Classification Report:
               precision    recall  f1-score   support

           0       0.88      0.93      0.91        89
           1       0.74      0.98      0.84        89
           2       1.00      0.62      0.77        90

    accuracy                           0.84       268
   macro avg       0.87      0.84      0.84       268
weighted avg       0.87      0.84      0.84       268


🔷 Decision Tree Performance
Accuracy : 0.8881
Confusion Matrix:
 [[81  8  0]
 [ 3 86  0]
 [ 9 10 71]]
Classification Report:
               precision    recall  f1-score   support

           0       0.87      0.91      0.89        89
           1       0.83      0.97      0.89        89
           2       1.00      0.79      0.88        90
```

```
    accuracy                           0.89       268
   macro avg       0.90      0.89      0.89       268
weighted avg       0.90      0.89      0.89       268


🔷 Random Forest Performance
Accuracy : 0.8881
Confusion Matrix:
 [[86  3  0]
 [ 4 81  4]
 [ 9 10 71]]
Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.97      0.91        89
           1       0.86      0.91      0.89        89
           2       0.95      0.79      0.86        90

    accuracy                           0.89       268
   macro avg       0.89      0.89      0.89       268
weighted avg       0.89      0.89      0.89       268
```